

Rajalakshmi Engineering College

Name: Naren Kartic B
Email: 241901065@rajalakshmi.edu.in
Roll no:
Phone: 6374678252
Branch: REC
Department: CSE (CS) - Section 1
Batch: 2028
Degree: B.E - CSE (CS)

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 7_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement:

Rathish is planning a road trip and needs a program to convert speeds between miles per hour (MPH) and kilometers per hour (KPH).

Create an interface, SpeedConverter, with a method convertSpeed(double mph). Implement the interface with MPHtoKPHConverter class, allowing Rathish to input MPH and receive the converted speed in KPH, rounded to two decimal points.

Formula: Speed in KPH = 1.60934 * Speed in MPH.

Input Format

The input consists of a single double-point number representing the speed in miles per hour (MPH).

Output Format

The output displays the converted speed (double-point number) in kilometers per hour (KPH) rounded off to two decimal points in the following format:

"Speed in KPH: <<converted speed>>".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1.0

Output: Speed in KPH: 1.61

Answer

```
import java.util.Scanner;

interface SpeedConverter {
    double convertSpeed(double mph);
}

class MPHtoKPHConverter implements SpeedConverter {
    public double convertSpeed(double mph) {
        return mph * 1.60934;
    }
}

class SpeedConversionApp {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double speedInMPH = scanner.nextDouble();

        SpeedConverter converter = new MPHtoKPHConverter();

        double speedInKPH = converter.convertSpeed(speedInMPH);

        System.out.printf("Speed in KPH: %.2f\n", speedInKPH);
    }
}
```

```
        scanner.close();
    }
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

A developer aims to create a budget management system using two interfaces, ExpenseRecorder for recording expenses and BudgetCalculator for calculating remaining budgets.

The ExpenseTracker class implements these interfaces, allowing users to input an initial budget and record expenses iteratively until entering 0.0 as a sentinel value.

The program then computes and displays the remaining budget or notifies of budget exceedance.

Example

Input

100.0

20.0 30.0 10.0 0.0

Output

Remaining budget: Rs. 40.00

Explanation

The initial budget is 100.0. Expenses of 20.0, 30.0, and 10.0 are recorded.

Remaining budget is calculated ($100.0 - 20.0 - 30.0 - 10.0 = 40.0$).

Input Format

The first line of input is the initial budget as a double-point number (double type). The budget is a positive number.

The second line of input consists of individual expenses as double-point numbers. Each expense is separated by space.

To end the input, an expense of 0.0 is used.

Output Format

The output displays the remaining budget, formatted to two decimal places, in the following format:

If the remaining budget (double type) is non-negative, it prints "Remaining budget: Rs. [remainingBudget]".

If the remaining budget is negative, it prints "No remaining budget, You've exceeded your budget!".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 100.0

20.0 30.0 10.0 0.0

Output: Remaining budget: Rs. 40.00

Answer

```
import java.util.Scanner;
```

```
interface ExpenseRecorder {  
    void recordExpense(double expense);  
}
```

```
interface BudgetCalculator {  
    double calculateRemainingBudget();  
}
```

```
class ExpenseTracker implements ExpenseRecorder, BudgetCalculator {  
    private double budget;  
    private double totalExpenses;
```

```

public ExpenseTracker(double budget) {
    this.budget = budget;
    this.totalExpenses = 0.0;
}

public void recordExpense(double expense) {
    totalExpenses += expense;
}

public double calculateRemainingBudget() {
    return budget - totalExpenses;
}
}

class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        double budget = scanner.nextDouble();

        ExpenseTracker tracker = new ExpenseTracker(budget);

        double expense;
        do {
            expense = scanner.nextDouble();
            tracker.recordExpense(expense);
        } while (expense != 0.0);

        double remainingBudget = tracker.calculateRemainingBudget();
        if (remainingBudget >= 0) {
            System.out.printf("Remaining budget: Rs. %.2f", remainingBudget);
        } else {
            System.out.println("No remaining budget, You've exceeded your
budget!");
        }
    }
}

```

Status : Correct

Marks : 10/10

3. Problem Statement:

Sam is developing a geometry application and needs a class for trapezoid calculations. Create a "Trapezoid" class implementing a "ShapeInput" interface with a method to input trapezoid dimensions.

Also, implement a "ShapeCalculator" interface with methods to compute area and perimeter. In the "Main" class, instantiate Trapezoid, gather user input, and display the calculated area and perimeter with two decimal places.

Note

Area of Trapezoid = $(1/2) * (base1 + base2) * height$

Perimeter of Trapezoid = $base1 + base2 + side1 + side2$

Input Format

The first line of input is a double-point value representing base1 of the trapezoid.

The second line of input is a double-point value representing base2 of the trapezoid.

The third line of input is a double-point value representing the height of the trapezoid.

The fourth line of input is a double-point value representing side1 of the trapezoid.

The fifth line of input is a double-point value representing side2 of the trapezoid.

Output Format

The output displays the two lines of the calculated area (double type) and perimeter (double type) of the trapezoid, each rounded to two decimal places in the following format:

"Area of the Trapezoid: <<calculated area>>".

Perimeter of the Trapezoid: <<calculated perimeter>>".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1.0

2.0

1.0

3.0

1.0

Output: Area of the Trapezoid: 1.50

Perimeter of the Trapezoid: 7.00

Answer

```
import java.util.Scanner;
```

```
interface ShapeInput {  
    void getInput();  
}
```

```
interface ShapeCalculator {  
    double calculateArea();  
    double calculatePerimeter();  
}
```

```
class Trapezoid implements ShapeInput, ShapeCalculator {  
    private double base1, base2, height, side1, side2;
```

```
    public void getInput() {  
        Scanner scanner = new Scanner(System.in);
```

```
        base1 = scanner.nextDouble();  
        base2 = scanner.nextDouble();  
        height = scanner.nextDouble();  
        side1 = scanner.nextDouble();  
        side2 = scanner.nextDouble();  
    }
```

```
    public double calculateArea() {  
        return 0.5 * height * (base1 + base2);  
    }
```

```

    public double calculatePerimeter() {
        return base1 + base2 + side1 + side2;
    }
}

public class Main {
    public static void main(String[] args) {
        Trapezoid trapezoid = new Trapezoid();
        trapezoid.getInput();

        double area = trapezoid.calculateArea();
        double perimeter = trapezoid.calculatePerimeter();

        System.out.println("Area of the Trapezoid: " + String.format("%.2f", area));
        System.out.println("Perimeter of the Trapezoid: " + String.format("%.2f",
perimeter));
    }
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Jeevan is developing a fitness-tracking application to monitor daily physical activity.

The application incorporates a `FitnessTracker` class that implements two interfaces: `StepCounter` for tracking the number of steps taken and `CalorieCalculator` for estimating total calories burned based on total steps.

Jeevan needs your help creating a program.

Note

The calorie calculation formula is: $\text{Total caloriesBurned} = (\text{total steps} / 100.0) * 20.0$.

Input Format

The first line of input is an integer n , representing the number of days Jeevan

wants to input data.

The second line consists of space-separated integers, representing the number of steps Jeevan took on each day.

Output Format

The first line of output prints: "Total Steps: <totalSteps>", where '<totalSteps>' is the sum of steps (integer) taken over 'n' days.

The second line prints: "Calories Burned: <caloriesBurned>", where '<caloriesBurned>' is the estimated total calories (double-point number) burned based on the total steps taken rounded off to two decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

340 234 987

Output: Total Steps: 1561

Calories Burned: 312.20

Answer

```
import java.util.Scanner;
```

```
interface StepCounter {  
    void countSteps(int steps);  
}
```

```
interface CalorieCalculator {  
    double calculateCaloriesBurned(int steps);  
}
```

```
class FitnessTracker implements StepCounter, CalorieCalculator {  
    private int totalSteps;  
  
    public void countSteps(int steps) {  
        totalSteps += steps;  
    }  
}
```

```

    public double calculateCaloriesBurned(int steps) {
        double caloriesBurned = (steps / 100.0) * 20.0;
        return caloriesBurned;
    }

    public int getTotalSteps() {
        return totalSteps;
    }
}

class Main
{

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        FitnessTracker tracker = new FitnessTracker();

        int n = scanner.nextInt();

        for (int i = 0; i < n; i++) {
            int steps = scanner.nextInt();
            tracker.countSteps(steps);
        }

        int totalSteps = tracker.getTotalSteps();
        System.out.println("Total Steps: " + totalSteps);

        double caloriesBurned = tracker.calculateCaloriesBurned(totalSteps);
        System.out.printf("Calories Burned: %.2f%n", caloriesBurned);

        scanner.close();
    }
}

```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Naren Kartic B
Email: 241901065@rajalakshmi.edu.in
Roll no:
Phone: 6374678252
Branch: REC
Department: CSE (CS) - Section 1
Batch: 2028
Degree: B.E - CSE (CS)

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 7_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. Which of the following statements is true regarding default methods in Java interfaces?

Answer

A default method can be overridden in a class implementing the interface.

Status : Correct

Marks : 1/1

2. How can a class explicitly call a default method from an interface if there is a naming conflict?

Answer

Using InterfaceName.super.methodName();

Status : Correct

Marks : 1/1

3. What happens when an implementing class does not override a default method from an interface?

Answer

The default method's implementation from the interface will be used.

Status : Correct

Marks : 1/1

4. Can a Java interface contain both default and static methods?

Answer

Yes, an interface can have both default and static methods.

Status : Correct

Marks : 1/1

5. What is the output of the following code?

```
interface X {  
    default void show() {  
        System.out.println("X's Default Method");  
    }  
}
```

```
interface Y {  
    default void show() {  
        System.out.println("Y's Default Method");  
    }  
}
```

```
class Z implements X, Y {  
    public void show() {  
        System.out.println("Z's Method");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Z obj = new Z();  
    }  
}
```

```
        obj.show();
    }
}
```

Answer

Z's Method

Status : Correct

Marks : 1/1

6. What is the output of the following code?

```
interface A {
    static void display() {
        System.out.println("Static method in A");
    }
}
```

```
class B implements A {
    static void display() {
        System.out.println("Static method in B");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        B.display();
    }
}
```

Answer

Static method in B

Status : Correct

Marks : 1/1

7. What is the output of the following code?

```
interface A {
    default void show() {
        System.out.println("A's Default Method");
    }
}
```

```

    }
}

interface B {
    default void show() {
        System.out.println("B's Default Method");
    }
}

class C implements A, B {
    public void show() {
        A.super.show();
    }
}

public class Main {
    public static void main(String[] args) {
        C obj = new C();
        obj.show();
    }
}

```

Answer

A's Default Method

Status : Correct

Marks : 1/1

8. Which of the following statements about Java interfaces is true?

Answer

A class can implement multiple interfaces.

Status : Correct

Marks : 1/1

9. Which of the following is the correct way to declare an interface in Java?

Answer

```
interface Vehicle { void start();}
```

Status : Correct

Marks : 1/1

10. How do you call a static method from an interface MyInterface?

Answer

```
MyInterface.staticMethod();
```

Status : Correct

Marks : 1/1

11. What is the output of the following code?

```
interface A {  
    default void show() {  
        System.out.println("A's Default Method");  
    }  
}
```

```
class B {  
    public void show() {  
        System.out.println("B's Method");  
    }  
}
```

```
class C extends B implements A {  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.show();  
    }  
}
```

Answer

B's Method

Status : Correct

Marks : 1/1

12. What is the output of the following code?

```
interface MathOperations {  
    static int square(int x) {  
        return x * x;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        System.out.println(MathOperations.square(5));  
    }  
}
```

Answer

25

Status : Correct

Marks : 1/1

13. What is the primary purpose of static methods in Java interfaces?

Answer

They allow an interface to provide helper methods without requiring an implementing class.

Status : Correct

Marks : 1/1

14. Consider a class implementing an interface and extending a class, both having a method with the same name. Which method gets called?

Answer

The method from the superclass

Status : Correct

Marks : 1/1

15. If a class implements two interfaces that have the same default method, what must the class do?

Answer

The class must override the method to resolve ambiguity.

Status : Correct

Marks : 1/1

Rajalakshmi Engineering College

Name: Naren Kartic B
Email: 241901065@rajalakshmi.edu.in
Roll no:
Phone: 6374678252
Branch: REC
Department: CSE (CS) - Section 1
Batch: 2028
Degree: B.E - CSE (CS)

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 7_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 30

Section 1 : Coding

1. Problem Statement:

Alice has been tasked with implementing a simple calculator interface and a corresponding class for performing basic addition and subtraction operations. The task is to create an interface called Calculator with two methods: add and subtract. The add method should take two numbers as input and return their sum, while the subtract method should take two numbers as input and return their difference.

Implement a class called SimpleCalculator that implements the Calculator interface. This class should provide the functionality for adding and subtracting numbers. Write a code that satisfies the above requirements.

Input Format

The first line of input consists of a single integer, representing the choice

If the choice is 1 or 2, the next two lines consist of 2 double values, representing the numbers to do addition or subtraction.

Output Format

The output prints a float-value with one decimal value representing the sum of two number or difference of two number.

Refer to the sample output for format specification.

Sample Test Case

Input: 1

5.5

3.5

Output: Result: 9.0

Answer

```
import java.util.Scanner;
```

```
interface Calculator {  
    double add(double num1, double num2);  
    double subtract(double num1, double num2);  
}
```

```
class SimpleCalculator implements Calculator {  
    public double add(double num1, double num2) {  
        return num1 + num2;  
    }  
    public double subtract(double num1, double num2) {  
        return num1 - num2;  
    }  
}
```

```
class MathOperationsProgram {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        SimpleCalculator calculator = new SimpleCalculator();
```

```

int choice = scanner.nextInt();

if (choice == 1) {
    double num1 = scanner.nextDouble();
    double num2 = scanner.nextDouble();
    double result = calculator.add(num1, num2);
    System.out.println("Result: " + result);
} else if (choice == 2) {
    double num1 = scanner.nextDouble();
    double num2 = scanner.nextDouble();
    double result = calculator.subtract(num1, num2);
    System.out.println("Result: " + result);
} else {
    System.out.println("Invalid choice. Please choose 1 for addition or 2 for subtraction.");
}

scanner.close();
}
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Sophia is developing a matrix analysis tool for a data analytics company. The tool needs to analyze square matrices and extract insights from the matrix diagonals.

To organize the code properly, Sophia creates an interface named Matrix that declares a method for finding the smallest and largest elements along the principal and secondary diagonals of the matrix.

Sophia then creates a class named MatrixAnalyzer that implements the Matrix interface. This class provides the logic to process a given square matrix and print:

The smallest and largest elements in the principal diagonal (from top-left to bottom-right). The smallest and largest elements in the secondary

diagonal (from top-right to bottom-left).

Your task is to implement the Matrix interface and the MatrixAnalyzer class. The main driver program (in the class Main) will read the input matrix, create an instance of MatrixAnalyzer, and invoke its method to display the results.

Input Format

The first line contains an integer n, representing the size of the square matrix.

The next n lines each contain n integers separated by spaces, representing the elements of the matrix.

Output Format

The output prints the four lines:

"Smallest Element - 1: <smallest element in the principal diagonal>" (integer)

"Largest Element - 1: <largest element in the principal diagonal>" (integer)

"Smallest Element - 2: <smallest element in the secondary diagonal>" (integer)

"Largest Element - 2: <largest element in the secondary diagonal>" (integer)

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

7 8 9 0 1

2 3 4 5 6

5 4 2 0 8

23 5 6 8 9

12 5 6 7 32

Output: Smallest Element - 1: 2

Largest Element - 1: 32

Smallest Element - 2: 1

Largest Element - 2: 12

Answer

```

import java.util.Scanner;

interface Matrix {
    void diagonalsMinMax(int[][] matrix);
}

class MatrixAnalyzer implements Matrix {
    public void diagonalsMinMax(int[][] matrix) {
        int n = matrix.length;
        int minPrincipal = matrix[0][0];
        int maxPrincipal = matrix[0][0];
        int minSecondary = matrix[0][n - 1];
        int maxSecondary = matrix[0][n - 1];

        for (int i = 0; i < n; i++) {
            if (matrix[i][i] < minPrincipal) minPrincipal = matrix[i][i];
            if (matrix[i][i] > maxPrincipal) maxPrincipal = matrix[i][i];
            if (matrix[i][n - 1 - i] < minSecondary) minSecondary = matrix[i][n - 1 - i];
            if (matrix[i][n - 1 - i] > maxSecondary) maxSecondary = matrix[i][n - 1 - i];
        }

        System.out.println("Smallest Element - 1: " + minPrincipal);
        System.out.println("Largest Element - 1: " + maxPrincipal);
        System.out.println("Smallest Element - 2: " + minSecondary);
        System.out.println("Largest Element - 2: " + maxSecondary);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[][] matrix = new int[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }
        MatrixAnalyzer analyzer = new MatrixAnalyzer();
        analyzer.diagonalsMinMax(matrix);
    }
}

```

3. Problem Statement

Develop a program for managing employee information that caters to both full-time and part-time employees. The program should be capable of computing the salary for each category of employee and presenting their particulars. To achieve this, create two classes, FullTimeEmployee and PartTimeEmployee, that adhere to the Employee interface.

The program is expected to accept input data, including the name and monthly salary for full-time employees, as well as the name, hourly rate, and hours worked for part-time employees. Subsequently, it should calculate and exhibit the employee details and their respective salaries.

For Full-Time employees, the annual salary should be calculated as 12 times the monthly salary.

For Part-Time employees, the salary calculation should be based on the formula: hourly rate * hours worked.

Input Format

The first line of input should be a string representing the name of a full-time employee.

The second line of input should be an integer representing the monthly salary of the full-time employee.

The third line of input should be a string representing the name of a part-time employee.

The fourth line of input should be an integer representing the hourly rate of the part-time employee.

The fifth line of input should be an integer representing the number of hours worked by the part-time employee.

Output Format

The output displays the following details:

Full-Time Employee Details:

Name: [Full-Time Employee Name] (string)

Monthly Salary: \$[Monthly Salary] (integer)

Annual Salary: \$[12 times Monthly Salary] (integer)

Part-Time Employee Details:

Name: [Part-Time Employee Name] (string)

Hourly Rate: \$[Hourly Rate] (integer)

Hours Worked: [Hours Worked] hours (integer)

Monthly Salary: \$[Calculated Monthly Salary] (integer)

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: John Smith

15000

Mary Johnson

100

100

Output: Full-Time Employee Details:

Name: John Smith

Monthly Salary: \$15000

Annual Salary: \$180000

Part-Time Employee Details:

Name: Mary Johnson
Hourly Rate: \$100
Hours Worked: 100 hours
Monthly Salary: \$10000

Answer

```
import java.util.Scanner;
```

```
import java.util.Scanner;
```

```
interface Employee {  
    String getName();  
    double calculateSalary();  
    void displayDetails();  
}
```

```
class FullTimeEmployee implements Employee {  
    private String name;  
    private double monthlySalary;  
  
    public FullTimeEmployee(String name, double monthlySalary) {  
        this.name = name;  
        this.monthlySalary = monthlySalary;  
    }
```

```
    @Override  
    public String getName() {  
        return name;  
    }
```

```
    @Override  
    public double calculateSalary() {  
        return 12 * monthlySalary;  
    }
```

```
    @Override  
    public void displayDetails() {  
        System.out.println("Full-Time Employee:");  
        System.out.println("Name: " + name);  
        System.out.println("Annual Salary: $" + String.format("%.2f",  
calculateSalary()));  
    }
```

```
}
```

```
class PartTimeEmployee implements Employee {  
    private String name;  
    private double hourlyRate;  
    private double hoursWorked;  
  
    public PartTimeEmployee(String name, double hourlyRate, double  
hoursWorked) {  
        this.name = name;  
        this.hourlyRate = hourlyRate;  
        this.hoursWorked = hoursWorked;  
    }  
  
    @Override  
    public String getName() {  
        return name;  
    }  
  
    @Override  
    public double calculateSalary() {  
        return hourlyRate * hoursWorked;  
    }  
  
    @Override  
    public void displayDetails() {  
        System.out.println("Part-Time Employee:");  
        System.out.println("Name: " + name);  
        System.out.println("Salary: $" + String.format("%.2f", calculateSalary()));  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        // Full-Time Employee Input  
        System.out.print("Enter Full-Time Employee Name: ");  
        String ftName = scanner.nextLine();  
        System.out.print("Enter Full-Time Employee Monthly Salary: ");  
        double ftMonthlySalary = scanner.nextDouble();  
        scanner.nextLine(); // Consume newline  
    }  
}
```

```
FullTimeEmployee ftEmployee = new FullTimeEmployee(ftName,
ftMonthlySalary);
```

```
// Part-Time Employee Input
System.out.print("Enter Part-Time Employee Name: ");
String ptName = scanner.nextLine();
System.out.print("Enter Part-Time Employee Hourly Rate: ");
double ptHourlyRate = scanner.nextDouble();
System.out.print("Enter Part-Time Employee Hours Worked: ");
double ptHoursWorked = scanner.nextDouble();
scanner.nextLine(); // Consume newline
```

```
PartTimeEmployee ptEmployee = new PartTimeEmployee(ptName,
ptHourlyRate, ptHoursWorked);
```

```
System.out.println("\n--- Employee Details ---");
ftEmployee.displayDetails();
System.out.println();
ptEmployee.displayDetails();
```

```
scanner.close();
```

```
}
}
```

```
class EmployeeInheritanceDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String fullName = scanner.nextLine();
        int fullTimeSalary = scanner.nextInt();
        scanner.nextLine();
        String partTimeName = scanner.nextLine();
        int hourlyRate = scanner.nextInt();
        int hoursWorked = scanner.nextInt();
        FullTimeEmployee fullTimeEmployee = new FullTimeEmployee(fullName,
fullTimeSalary);
        PartTimeEmployee partTimeEmployee = new
PartTimeEmployee(partTimeName, hourlyRate, hoursWorked);
        fullTimeEmployee.displayDetails();
        System.out.println();
        partTimeEmployee.displayDetails();
        scanner.close();
    }
}
```

```
}
```

Status : **Wrong**

Marks : 0/10

4. Problem Statement

Oviya is fascinated by automorphic numbers and wants to create a program to determine whether a given number is an automorphic number or not.

An automorphic number is a number whose square ends with the same digits as the number itself. For example, $25 = (25)^2 = 625$

Oviya has defined two interfaces: NumberInput for taking user input and AutomorphicChecker for checking if a given number is automorphic. The class AutomorphicNumber implements both interfaces.

Help her complete the task.

Input Format

The input consists of a single integer n.

Output Format

If the input number is an automorphic number, print "n is an automorphic number". Otherwise, print "n is not an automorphic number".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 25

Output: 25 is an automorphic number

Answer

```
import java.util.Scanner;
```

```

interface NumberInput {
    int getInput();
}

interface AutomorphicChecker {
    boolean checkAutomorphic(int num);
}

class AutomorphicNumber implements NumberInput, AutomorphicChecker {
    private int number;

    public int getInput() {
        Scanner scanner = new Scanner(System.in);
        return scanner.nextInt();
    }

    public boolean checkAutomorphic(int num) {
        int square = num * num;
        while (num > 0) {
            if (num % 10 != square % 10)
                return false;
            num = num / 10;
            square = square / 10;
        }
        return true;
    }
}

public class Main {
    public static void main(String[] args) {
        AutomorphicNumber automorphicNumber = new AutomorphicNumber();
        int inputNumber = automorphicNumber.getInput();

        boolean isAutomorphic =
        automorphicNumber.checkAutomorphic(inputNumber);

        if (isAutomorphic) {
            System.out.println(inputNumber+" is an automorphic number");
        } else {
            System.out.println(inputNumber+" is not an automorphic number");
        }
    }
}

```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Naren Kartic B
Email: 241901065@rajalakshmi.edu.in
Roll no:
Phone: 6374678252
Branch: REC
Department: CSE (CS) - Section 1
Batch: 2028
Degree: B.E - CSE (CS)

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 7_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 0

Section 1 : Coding

1. Problem Statement:

Rajiv is analyzing the energy consumption in his household and wants to calculate the total cost based on the daily energy usage. He is given the rate per unit of electricity and the energy consumed for multiple days. To structure this calculation efficiently, he decides to use an interface-based approach.

Implement an interface CostCalculator with the necessary methods to retrieve energy details and compute the cost. The calculations should be handled in the EnergyConsumptionTracker class, while the EnergyConsumptionApp class should only handle input and output.

Formula

Energy Cost for one day = Energy Consumed per day * Rate Per Unit

Input Format

The first line of input consists of the rate per unit as an 'R' (a double value).

The second line of input consists of the number of days 'N' (an integer).

The third line of input consists of the daily energy consumption values for each day 'D' (double values), separated by space.

Output Format

The first line of the output prints: "Day-wise Energy Cost:"

The next N lines of the output print the day-wise energy costs(double type) and the total energy cost (double type) in Indian Rupees in the following format: "Day [day_number]: Rs. [energy_cost]"

The last line of the output prints: "Total Energy Cost: Rs. [total_cost]"

Note: energy_cost and total_cost are rounded off to two decimal points

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 0.01

3

10.0 20.0 30.0

Output: Day-wise Energy Cost:

Day 1: Rs. 0.10

Day 2: Rs. 0.20

Day 3: Rs. 0.30

Total Energy Cost: Rs. 0.60

Answer

```
import java.util.Scanner;
```

```
import java.util.Scanner;
```



```
interface CostCalculator {  
    double getRatePerUnit();  
    int getEnergyConsumed(int day);  
    double calculateTotalCost(int numDays);  
}
```

```
class EnergyConsumptionTracker implements CostCalculator {  
    private double ratePerUnit;  
    private int[] dailyEnergyConsumed;  
  
    public EnergyConsumptionTracker(double ratePerUnit, int numDays) {  
        this.ratePerUnit = ratePerUnit;  
        this.dailyEnergyConsumed = new int[numDays];  
    }
```

```
    @Override  
    public double getRatePerUnit() {  
        return ratePerUnit;  
    }
```

```
    @Override  
    public int getEnergyConsumed(int day) {  
        if (day >= 0 && day < dailyEnergyConsumed.length) {  
            return dailyEnergyConsumed[day];  
        }  
        return 0; // Or throw an exception for invalid day  
    }
```

```
    public void setEnergyConsumed(int day, int energy) {  
        if (day >= 0 && day < dailyEnergyConsumed.length) {  
            this.dailyEnergyConsumed[day] = energy;  
        }  
    }
```

```
    @Override  
    public double calculateTotalCost(int numDays) {  
        double totalCost = 0;  
        for (int i = 0; i < numDays; i++) {  
            totalCost += dailyEnergyConsumed[i] * ratePerUnit;  
        }  
        return totalCost;  
    }
```

```

    }
}

class EnergyConsumptionApp {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double ratePerUnit = scanner.nextDouble();
        int numDays = scanner.nextInt();

        EnergyConsumptionTracker tracker = new
        EnergyConsumptionTracker(ratePerUnit, numDays);

        for (int i = 0; i < numDays; i++) {
            int energyConsumed = scanner.nextInt();
            tracker.setEnergyConsumed(i, energyConsumed);
        }

        System.out.printf("%.2f%n", tracker.calculateTotalCost(numDays));

        scanner.close();
    }
}

```

```

class EnergyConsumptionApp {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double ratePerUnit = scanner.nextDouble();
        int numDays = scanner.nextInt();

        CostCalculator tracker = new EnergyConsumptionTracker(ratePerUnit,
        numDays);

        tracker.getEnergyDetails(scanner);
        tracker.calculateAndDisplayCost();

        scanner.close();
    }
}

```

Status : Wrong

Marks : 0/10

Rajalakshmi Engineering College

Name: Naren Kartic B
Email: 241901065@rajalakshmi.edu.in
Roll no:
Phone: 6374678252
Branch: REC
Department: CSE (CS) - Section 1
Batch: 2028
Degree: B.E - CSE (CS)

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 7_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Jaheer is working on a health monitoring system to help individuals calculate their Body Mass Index (BMI). He has implemented a basic BMI calculator and an interface called HealthCalculator. It should have a method called calculateBMI.

You are tasked with creating a program that takes weight and height as input, calculates the BMI using the BMI Calculator class, and displays the result. If the height or weight is less than or equal to zero, then return -1.

Formula: $BMI = \text{weight} / (\text{height} * \text{height})$

Input Format

The first line of input consists of a double value W, the person's weight in kilograms.

The second line consists of a double value H, the height of the person in meters.

Output Format

The output displays "BMI: " followed by a double value, representing the calculated BMI, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 70.0

1.75

Output: BMI: 22.86

Answer

```
import java.util.Scanner;

interface HealthCalculator {
    double calculateBMI(double weight, double height);
}

class BMICalculator implements HealthCalculator {
    public double calculateBMI(double weight, double height) {
        if (weight <= 0 || height <= 0) {
            return -1.0;
        }
        double bmi = weight / (height * height);
        return bmi;
    }
}

class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double weight = scanner.nextDouble();
        double height = scanner.nextDouble();

        BMICalculator bmiCalculator = new BMICalculator();
```

```
double bmi = bmiCalculator.calculateBMI(weight, height);  
  
System.out.printf("BMI: %.2f\n", bmi);  
  
    scanner.close();  
}  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Naren Kartic B
Email: 241901065@rajalakshmi.edu.in
Roll no:
Phone: 6374678252
Branch: REC
Department: CSE (CS) - Section 1
Batch: 2028
Degree: B.E - CSE (CS)

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 7_Q3

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

A financial analyst, Alex, needs a program to calculate simple interest for various financial transactions. He requires a straightforward tool that takes in the principal amount, interest rate, and time in years and computes the interest.

The formula to be used is: $\text{Interest} = \text{Principal} \times \text{Rate} \times \text{Time} / 100$

Implement this functionality using the InterestCalculator interface and the SimpleInterestCalculator class.

Input Format

The first line of input consists of the principal amount P as a double value.

The second line of input consists of the annual interest rate r as a double value.

The third line of input consists of the number of years t as a positive integer, which is an integer value.

Output Format

The output displays the calculated simple interest in the following format: "Simple Interest: [interest_value]", Here, [interest_value] should be replaced with the actual interest value calculated by the program.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1000.00

5.00

2

Output: Simple Interest: 100.0

Answer

```
import java.util.Scanner;
```

```
interface InterestCalculator {  
    double simpleInterest(double principal, double rate, int time);  
}
```

```
class SimpleInterestCalculator implements InterestCalculator {  
    public double simpleInterest(double principal, double rate, int time) {  
        double interest = (principal * rate * time) / 100;  
        return interest;  
    }  
}
```

```
class Main {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        double principal = scanner.nextDouble();  
  
        double rate = scanner.nextDouble();
```

```
int time = scanner.nextInt();  
  
InterestCalculator calculator = new SimpleInterestCalculator();  
  
double interest = calculator.simpleInterest(principal, rate, time);  
  
System.out.println("Simple Interest: " + interest);  
  
    }  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Naren Kartic B
Email: 241901065@rajalakshmi.edu.in
Roll no:
Phone: 6374678252
Branch: REC
Department: CSE (CS) - Section 1
Batch: 2028
Degree: B.E - CSE (CS)

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 7_Q4

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Maria, a software developer, is working on an inventory management system project using Java that utilizes an inventory interface to manage a store's products.

The interface should define two methods: `addProduct`, which adds a product by accepting its name, price, and quantity, and `calculateTotalValue`, which computes the total value of all products in the inventory. Implement the interface in a class called `SimpleInventory`, which internally manages a list of `Product` objects.

Each `Product` object should encapsulate the product's name, price, and quantity and include a method to calculate its value as $\text{price} \times \text{quantity}$. The system should allow users to dynamically add products to the inventory and calculate the total value of all products stored.

Help Maria achieve the task.

Input Format

The first line of input consists of an integer to choose one of the following options:

- 1 - to add a product to the inventory.
- 2 - to calculate and view the total inventory value.
- 3 - to exit the program.

For Choice 1 (Add Product):

The next input line is the string representing the product name as a string (single or multi-word, without quotes).

The next line is a double value representing the price as a decimal value

The next line is an integer value representing the quantity as an integer

For Choices 2 and 3, no additional input is required

Output Format

The output displays the results of the commands as follows:

- For the addProduct command, the program should display "Product added to inventory."
- For choice 2, the program should display "Total inventory value [totalvalue].
"The total value should be displayed with one decimal place. If there is no product in the inventory, print the total as 0.0.
- For choice 3, the program should exit

If the choice is not 1, 2, or 3, then print "Invalid choice. Please select a valid option (1/2/3).".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1

Laptop

800.0

3

2

5

3

Output: Product added to inventory.

Total inventory value: \$2400.0

Invalid choice. Please select a valid option (1/2/3).

Answer

```
import java.util.Scanner;
```

```
interface Inventory {  
    void addProduct(String productName, double price, int quantity);  
    double calculateTotalValue();  
}
```

```
class SimpleInventory implements Inventory {  
    private Product[] products;  
    private int count;
```

```
    public SimpleInventory(int size) {  
        products = new Product[size];  
        count = 0;  
    }
```

```
    public void addProduct(String productName, double price, int quantity) {  
        if (count < products.length) {  
            products[count] = new Product(productName, price, quantity);  
            count++;  
            System.out.println("Product added to inventory.");  
        } else {  
            System.out.println("Inventory full. Cannot add more products.");  
        }  
    }
```

```
    public double calculateTotalValue() {  
        double totalValue = 0;  
        for (int i = 0; i < count; i++) {
```

```

        totalValue += products[i].getValue();
    }
    return totalValue;
}

class Product {
    private String name;
    private double price;
    private int quantity;

    public Product(String name, double price, int quantity) {
        this.name = name;
        this.price = price;
        this.quantity = quantity;
    }

    public double getValue() {
        return price * quantity;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Inventory inventory = new SimpleInventory(10);
        while (true) {
            int choice = scanner.nextInt();
            if (choice == 1) {
                scanner.nextLine();
                String productName = scanner.nextLine();
                double price = scanner.nextDouble();
                int quantity = scanner.nextInt();
                inventory.addProduct(productName, price, quantity);
            } else if (choice == 2) {
                double totalValue = inventory.calculateTotalValue();
                System.out.println("Total inventory value: $" + totalValue);
            } else if (choice == 3) {
                break;
            } else {
                System.out.println("Invalid choice. Please select a valid option
(1/2/3).");
            }
        }
    }
}

```

```
    }  
    scanner.close();  
  }  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Naren Kartic B
Email: 241901065@rajalakshmi.edu.in
Roll no:
Phone: 6374678252
Branch: REC
Department: CSE (CS) - Section 1
Batch: 2028
Degree: B.E - CSE (CS)

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 7_Q5

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Raj is curious about how old he is in the current year.

He has asked you to create a simple program that calculates a person's age based on their birth year. You decide to implement this functionality using the AgeCalculator interface and the HumanAgeCalculator class.

Note: The current year is 2024. Calculate the current age by using the formula: current year - birth year.

Input Format

The input consists of an integer representing the birth year.

Output Format

The output displays "You are X years old." where X is an integer representing the calculated age based on the entered birth year.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1934

Output: You are 90 years old.

Answer

```
import java.util.Scanner;

interface AgeCalculator {
    int calculateAge(int birthYear);
}

class HumanAgeCalculator implements AgeCalculator {
    public int calculateAge(int birthYear) {
        int currentYear = 2024;
        return currentYear - birthYear;
    }
}

class AgeCalculatorApp {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        AgeCalculator ageCalculator = new HumanAgeCalculator();

        int birthYear = scanner.nextInt();
        int age = ageCalculator.calculateAge(birthYear);

        System.out.println("You are " + age + " years old.");
    }
}
```

Status : Correct

Marks : 10/10